

SPARQ: Scalable Privacy-preserving Aggregate Range Queries

Mahdiah Heidari pour

Augusta University

mheidari pour@augusta.edu

Maryam Rezapour

University of Connecticut

maryam.rezapour@uconn.edu

Benjamin Fuller

University of Connecticut

benjamin.fuller@uconn.edu

Hoda Maleki

Augusta University

hmaleki@augusta.edu

Gagan Agrawal

University of Georgia

gagrawal@uga.edu

Abstract

Searchable encryption enables efficient querying over outsourced data while limiting server-side leakage. Existing methods for associative aggregation queries on multidimensional array data suffer from significant limitations:

1. Use range structures that precompute ranges yielding either:
 - (a) Storage size proportional to the total domain size (Demertzis et al., TODS 2018/Falzon et al., PVLDB2022).
 - (b) Or linear-time search in the worst case (Chang et al., IEEE TKDE 2021).
2. Bucket-based methods (Espiritu, Markatou, Tamassia, PoPETS 2022) are only practical when the largest bucket is small.
3. Fully homomorphic encryption (FHE) approaches (Kushnir, Moskowich, Shaul, PoPETS 2024) remain computationally impractical.

Due to strong attacks on range schemes with access pattern leakage (Falzon et al., CCS 2020), we consider the task of oblivious search. We propose *oblivious segment trees*, which offer efficient storage and search. Our approach achieves polylogarithmic query complexity. Storage depends on the number of distinct values per dimension, not on the domain size. Under typical data distributions, segment trees are provably shorter than standard alternatives.

We evaluate performance on datasets with 1–3 dimensions and up to 5 million entries. Across multidimensional datasets, our method reduces the number of stored items by 10 to 10^{10} compared to prior non-FHE approaches. On a 32-thread machine, server-side query processing remains under 1.2 seconds.

1 Introduction

Scientific simulations and experiments store static data in large-scale multidimensional arrays [64, 71]. Outsourcing such data and analytics to third-party cloud providers is common [36, 37]. A client wishes to outsource a database $\mathcal{DB}$ to an honest but curious server.

The goal of searchable encryption is to simultaneously allow efficient querying to the outsourced data while limiting what can be learned by the server, called *leakage*. For a full review of searchable encryption, see previous surveys [8, 27, 42]. Prior attacks show how to recover either database or query contents from systems with “too much” leakage [12, 23, 24, 31–33, 41–46, 53, 69]. Attacks on range structures are particularly effective due to their geometric structure [23, 32, 43]. Due to these attacks, *we focus on high security solutions that leak only global statistics of the stored data*. Generally, volume leakage is removed by padding [19], such techniques are applicable in our solution.

Aggregation queries allow a client to compute a summary statistic over a subset of their multi-dimensional range data. Aggregation queries are frequently used in data analytics [36] such as the US census which reports differentially-private counts of demographics at multiple location granularities [1]. Standard aggregate queries are commonly used functions that group data tuples and compute a single value for each group including SUM, AVG, COUNT, MIN, MAX, and STDDEV [71]. In this work, we restrict our attention to aggregate queries

with associative properties. By associative, we mean that for data subgroups X and Y where $X \cap Y = \emptyset$ given $f(X), f(Y), \text{COUNT}(X), \text{COUNT}(Y)$ one can compute $f(X \cup Y)$. Many applications can be built from associative queries [30], examples include 1) studying disease risk by age, geographic region, and time and 2) studying temperature variation over time in geographic areas.

We briefly discuss non-associative statistics such as **MODE** and **MEDIAN** at the end of Section 3. We consider the setting of *sparse, multidimensional, static* data. Sparse means that not every tuple exists in the $\mathcal{DB}$. We target static analytical workloads, such as scientific datasets, where setup is a one-time cost amortized over many queries. SPARQ is designed for low-dimensional sparse aggregation workloads; our evaluation therefore focuses on 1D–3D datasets, which already cover many practical analytical settings.

1.1 Secure Range Structures for Aggregation

We consider a d -dimensional domain in which the value along dimension i lies in $[1, m_i]$ for each $1 \leq i \leq d$. Other numeric domains are translated to positive integers by normalization. (Table 1 shows common notation used throughout.) A *map* allows one to associate each range as a keyword in the map with a value of the aggregation output. A naive solution stores every possible range in the map. This solution stores $\ell = \prod_{i=1}^d m_i^2$ items in the map, revealing two inefficiencies:

1. Storage scales with the domain of each dimension.
2. Storage scales with the square of domain size.

Range data structures Demertzis et al. [20] eliminate the need for storage quadratic in the domain for **SUM**, **MIN**, and **MAX**. The idea is based on prefix sums. To illustrate, we consider a single dimension first. For map M , instead of storing all possible ranges one stores

$$M[[1, k]] = f([1, k]), 1 \leq k \leq m_1.$$

Then for **SUM** one can compute

$$f([i, j]) = M[[1, j]] - M[[1, i]].$$

Similarly, if one considers two dimensions one stores

$$M[[1, j] \times [1, k]] = f([1, j] \times [1, k]), 1 \leq j \leq m_1, 1 \leq k \leq m_2$$

and can compute $f([i_1, j_1] \times [i_2, j_2])$ using four queries to the map. This solution requires storage proportional to domain size, $m := \prod_{i=1}^d m_i$.¹ Demertzis et al. present more complex solutions for **MIN** and **MAX** with storage proportional to $\prod_{i=1}^d m_i \log m_i$.²

Falzon et al. [24] create trees that subdivide the domain recursively and then use range covers to look up nodes from this tree. One can associate the aggregated value with each node, supporting all associative aggregation functions. All three of their schemes produce trees at least as large as Demertzis et al.’s prefix arrays (see Table 2).

Section 1.3 covers two more works that require an oblivious multimap and fully-homomorphic encryption (FHE). Prior work is summarized in Table 2.

Cryptographic Storage of the Range Structures We focus on a single-server setting. In the single-server setting, the map can be stored using an oblivious map [6, 63, 70] requiring rounds proportional to the log of the size of the stored map using PathORAM [67] (or for the client to cache results which can grow prohibitive [28]). Alternatively, the map can be converted into a prefix array. The prefix array can be stored using prior work from database aggregation statistics in the multi-server setting. These works use techniques from distributed private information retrieval (PIR) [48], oblivious pseudorandom functions (OPRF) [26], (Incremental) Distributed Point Function (IDPF/DPF), and Function Secret Sharing (FSS) [9, 10, 29]. Often these systems are designed in the context of private heavy-hitters [7, 49, 56, 57].

¹ $f([i_1, j_1] \times [i_2, j_2]) = M[[1, j_1] \times [1, j_2]] - M[[1, i_1 - 1] \times [1, j_2]] - M[[1, j_1] \times [1, i_2 - 1]] + M[[1, i_1 - 1] \times [1, i_2 - 1]]$. One can extend this technique to handle a query in d dimensions using 2^d parallel calls to the M .

²Demertzis et al. [20] also presented two schemes to compute range minimum query utilizing *sparse table* techniques [4], one with $|M| = m \log m$ and the other with $|M| = m + N \log N$ where N is the size of the database. The latter requires two sequential accesses to oblivious maps.

Notation	Meaning
$\mathcal{DB}$	Database
N	$ \mathcal{DB} $
m_i	Maximum value of dimension i
m	$\prod_{i=1}^d m_i$
n_i	Number of distinct values in dimension i
n	$\prod_{i=1}^d n_i$
ℓ	Number of stored items
$\mathcal{R}$	Associated record with each value (over $\mathbb{R}$)
f	Aggregation applied (associative)

Table 1: Summary of Notation. We consider a value in $\mathcal{R}$, schemes extend to multiple aggregated values.

	Aggr.	Stored items	Total Queries	Seq.
O. Seg.	Assoc.	$2^d n$	$2 \prod_{i=1}^d 2 \log n_i$	$\log n$
Quad. [24]	Assoc.	$m + N^2$	1	1
Lin. [24]	Assoc.	$m + N$	R	1
Range Cov. [24]	Assoc.	$m + N \log^d m$	$\log^d R$	1
DPPDGP	SUM	m	2^d	1
[20]	MIN ₁	$m \log^d m$	2^d	1
	MIN ₂	$m + N \log^d N$	2^{d+1}	2
EMT [21]	SUM	$m^\alpha + N$	2^d	1
	MIN	m	2^d	1
KMS [47]	Assoc.	$N \log^{d-1} N$	1	1

Table 2: All systems instantiated as above only leak size information during setup and search. We let $m = \prod_{i=1}^d m_i$, $N = |\mathcal{DB}|$ and $n = \prod_{i=1}^d n_i$ where n_i is the number of distinct values in dimension i and m_i is the domain size of dimension i . $\epsilon > 0$ is a constant and t is time to homomorphically evaluate a comparison, current 25 seconds [68]. EMT and KMS are covered in Section 1.3. EMT SUM uses an oblivious multimap while DPPDBP [20], FMET [24], and other EMT [21] schemes require an oblivious map. EMT [21] SUM scheme is an α approximation where $0 < \alpha < 1$ is some constant. KMS [47] uses FHE so items are stored under FHE encryption not in a map, single round with computation of $t \cdot N^{1-\frac{1}{d}+\epsilon} + N^{1+\epsilon}$. R is the range size, note FMET [24] schemes can reveal the size of the range, for an oblivious version, one considers artificially divides the range so that $R = m$.

Whether using a single server or multiple servers, the size of stored data is prohibitive for multidimensional sparse data. The $\log_{10}$ of these sizes (without any cryptographic overhead) are shown in Table 3 under the column m . Having storage scale with m is impractical; the number of items to be stored often exceeds 10^{10} .

We ask **can one construct oblivious range aggregation structures whose storage is independent of domain size?**

1.2 Our Contribution

Unprotected aggregate and range structures typically use trees where queries 1) start at the root, 2) traverse to relevant nodes, and 3) aggregate values of some accessed nodes. Prior work [35, 62, 70] shows such trees can be efficiently encoded using oblivious RAM (ORAM). The key insight is that parent nodes are always accessed before their children, allowing the use of non-recursive ORAM, storing the current positions of children rather than their logical node identifiers. Updating these positions can be done with a single extra round, retaining round complexity that scales with the height of the tree. Recent work [2, 59] extends this approach to low-degree graphs.

The main drawback of using ORAM in comparison to alternatives like PIR or DPF is its higher round

d	dataset	ST		Key Value Based						FHE		H
		Max	Act	[20] SUM	[20] Min	[24] Quad	[24] Lin	[24] R. Cover	[22] Bucket.	[47] C&R	S / Q	
1	books	4.2	4.2	5.9	8.1	7.8	5.9	5.9	4.0	3.9	14 / -	
	gowalla	7.0	7.0	7.7	9.0	13.4	7.0	7.9	7.4	6.7	24 / -	
2	Spitz	6.2	4.6	6.0	4.7	6.5	6.0	6.1	5.5	4.0	22 / 22	
	cali	6.6	5.5	6.0	7.7	8.5	6.0	6.6	6.2	5.2	22 / 22	
	gowalla	7.2	5.7	10.8	13.0	10.0	9.8	9.8	9.6	5.7	25 / 26	
	gowalla	8.1	6.1	11.0	13.2	11.0	10.9	10.9	9.7	6.0	28 / 28	
	synthetic	7.2	6.8	6.6	8.4	12.0	6.7	8.4	8.1	7.1	24 / 24	
	synthetic	7.2	6.9	12.0	14.3	12.3	12.0	12.0	9.7	7.1	24 / 24	
	synthetic	6.6	6.6	12.0	14.3	12.3	12.0	12.0	9.8	7.1	22 / 22	
	nh	6.3	5.2	5.4	7.3	7.2	5.4	6.9	6.0	5.5	21 / 21	
3	gowalla	6.4	5.7	6.0	8.0	8.7	6.0	7.8	7.4	6.4	23 / 24	
	synthetic	7.2	7.1	18.0	21.4	18.0	18.0	18.0	13.5	8.3	24 / 24	
	synthetic	8.1	7.6	18.0	21.4	21.0	21.0	21.0	13.3	8.3	27 / 27	
	synthetic	8.1	7.6	18.0	21.4	21.0	21.0	21.0	13.3	8.3	27 / 27	

Table 3: $\log_{10}$ of stored items and Height of Segment and Quad Trees. For Map and tree based solutions we list the number of items in the M. For the EMT approach which relies on an MM we list the number of stored items when padded to the maximum number of items associated with any keyword. ST.Max, is the size of segment tree size based on $\prod_{i=1}^d (2n_i - 1)$. ST.Act, is the size of the segment tree which is stored in OMAP in practice per dataset based on the experiments. S is the height of the segment tree, and Q is the height of the quadtree for 2D and octree for 3D, it is omitted for 1D as it is a segment tree. EMT [22] uses $\alpha = 0.5$. Details on the datasets are introduced in Table 7.

Primitive	Work	Index	Storage	Communication	Lookup	Search
IDPF	Poplar [7], Popstar [49], Doplar [16], PLASMA [57], Mastic [56]	Prefix Array	$\lambda \cdot \log \ell$	$\lambda \cdot \log \ell$	1	$\lambda \log \ell$
ORAM	Recursive Path-ORAM [70]	Binary Tree	$\lambda \ell \log \ell$	$\lambda \log^2 \ell$	$\lambda \log \ell$	$\lambda \log \ell$
	Optimal ORAM [3]		$\lambda \ell$	$\lambda \log \ell$	$\lambda \log \ell$	$\lambda \log \ell$
	Oblivious Map (OM) [11]		$\lambda \ell + \lambda \log \ell \log \lambda$	$\frac{\log^2 \ell}{\log \log \ell} + \log \ell \log \lambda$	$\frac{\log \ell}{\log \log \ell} + \log \lambda$	$\log \ell$
	Oblivious MultiMap (OMM) [6]		$\lambda \ell \rho \log(\ell \rho)$	$\lambda \log^2(\ell \cdot \rho)$	$\log(\ell \cdot \rho)$	$\lambda \log \ell$

Table 4: Comparison of primitives. ℓ : number of keywords/items; ρ : max values per key; λ : security parameter. All items are asymptotic with big O elided except rounds of IDPF. Lookup and Search columns are the number of rounds for lookup and search respectively.

complexity. However, the children to visit is data-dependent; sequential lookups are required. Sequential lookups remove ORAM’s disadvantage. Looking ahead to Table 5 while an ORAM solution requires multiple rounds for a single lookup, one can traverse the entire tree for asymptotically the same number of rounds. Current DPF-based schemes are largely multi-server, providing qualitatively different security model than the single-server setting we consider. Single-server DEPIR is not yet practical [50, 58]. It is viable to use a linear scan PIR, but the tree traversal would require adaptive lookups in the array.

Unlike tree-based maps where one traverses a single path, in range (aggregation) trees one collects and aggregates multiple nodes to achieve the final value [14, 61, 72]. When such a structure is stored in oblivious RAM, one needs to hide the traversal through the tree. There are three metrics of range tree performance:

Storage The number of nodes in the tree, which we treat a node as the basic unit of storage throughout the rest of the paper. We discuss packing of nodes into ORAM blocks in Section 4.1.

Height The height of the stored tree which determines the number of rounds of the resulting cryptography. (Assuming that one is able to simultaneously make all accesses at a single level.) The height of the stored tree may depend on some summary statistics of $\mathcal{DB}$.

Accesses The maximum number of nodes that are retrieved both at each level of the tree and overall. This impacts the bandwidth and computationally efficiency of the resulting cryptography.

Crypto Primitive	Rounds Per Lookup	Full Search Rounds
IDPF [7]	1	$\log \ell$
OPRF	1	$\log \ell$
PIR [48]	1	$\log \ell$
ORAM [3]	$\log \ell$	$\log^2 \ell$
O. Rooted Tree [70]	$\log \ell$	$\log \ell$

Table 5: Comparison of the number of rounds for lookup of different cryptographic primitives. Rounds are the main weakness of ORAM. However, traversing a segment tree requires $\log \ell$ sequential accesses. Traversing this tree structure in ORAM can be done in only $\log \ell$ rounds. ℓ is total number of stored items.

	Segment tree	k-d tree	Quad/Octree	R-tree
Storage	$2^d n$	$O(N)$	$O(N)$	$O(N/B)$
Height	$\sum_{i=1}^d \log_r n_i$	$O(N)$	$O(\log_{2^d} N)$	$O(\log_B N)$
Worst Case Access	$4 \left(1 + \sum_{i=1}^{d-1} 2^i \prod_{j=1}^i (\log_r(n_j) - 1) \right)$	$O(N)$	$O(N)$	$O(N)$
Balanced?	Yes	No	No	Yes

Table 6: Comparison of various spatial data structures for Multidimensional Range Aggregation. B denotes the maximum capacity of pages containing organized data points. Bounds for heights of quad and segment trees are provided in Appendix 3.1.2, and bounds on the number of accesses of the segment tree are proved in Section 3.1.

Table 6 compares common range aggregation structures.³ We show new bounds on the heights of quadtrees and segment trees and the number of accesses of segment trees (Section 3.1).

Unfortunately, for structures with storage $O(N)$ there are no known sublinear bounds on the maximum number of accesses including k-d trees, quad/oct/hyperoct trees,⁴ and R-trees. Recall one can clearly achieve linear storage and query by returning the entire encrypted dataset with each query. While the *average-case* performance of these structures is much better, the worst case performance is relevant for oblivious algorithms. As an example, Chang et al. propose an oblivious implementation of B-trees [14]. However, their structure has a linear worst case for number of accessed nodes, creating a tradeoff between leakage (or correctness) and efficiency.

We focus on segment trees [5, 17, 60]. A segment tree can be seen as the range tree of Falzon et al. [24] where instead of dividing the domain at each domain, one divides based on the median. In Theorem 3, we show upper bounds on accesses at each level for segment trees:

$$\# \text{Accessed} \leq \sum_{i=1}^d 2 \prod_{j=1}^i (2(\log n_i - 2)).$$

In the above, n_i is the number of distinct values in dimension i . We additionally show upper bounds for segment tree height and show under reasonable assumptions on data, segment tree height is less than quad tree height [25]. Looking ahead to our experimental results, the height of quad trees is at least the height of segment trees (Table 3).

We prototype *oblivious segment trees* where all tree nodes are stored in a single array that is stored in ORAM. Nodes store the position of their children, with the client retaining only the ORAM stash and the address of the tree root.

Security We access the maximum number of nodes at each time step. The only leakage of our scheme is:

- **Setup:** The size and height of the oblivious segment tree. These values can be padded as appropriate.

³Building an efficient balanced R-tree with minimum node visits during search is an ongoing research topic [34, 52]; currently, worst case analysis remains linear in $|\mathcal{DB}|$.

⁴We use quad tree to refer to quad tree and its higher dimensional variants known as octtrees and hyperoctrees.

- **Search:** We search for the maximum possible required nodes of the tree at each level. As shown in our bound on $\#Accessed$, this property depends on the number of distinct values in each dimension which also influences the size of the segment tree. The number of accesses per level can be padded to mitigate leakage on n_i .

This approach achieves practical storage and search efficiency. We don't discuss padding more; everything in our approach is compatible with standard padding approaches.

The size of the stored segment tree is at most $2^d n = \prod_{i=1}^d 2n_i$. Non-recursive has a constant storage overhead, meaning for sparse datasets storage is substantially smaller than m . As discussed above, segment trees do require one to make multiple accesses per tree level. This value scales in d and logarithmically with the distinct values in each dimension, n_i . This increases our computation and bandwidth to

$$\begin{aligned} \text{Our BW} &:= \#accesses \cdot \text{cost per access} \\ &\leq \left(\sum_{i=1}^d 2 \prod_{j=1}^i (2(\log n_j - 2)) \right) \cdot \log 2^d n, \\ \text{Demertzis BW} &\leq (2^d) \cdot \log^2 m. \end{aligned}$$

We make more accesses than Demertzis prefix tree for larger d . Our number of accesses scales with $\prod_{i=1}^d 2\log(n_i)$ while the prefix sum only scales with 2^d . In Table 9, we show our bandwidth (ST.BW) and the bandwidth of prior work (Demertzis BW). The ratio is approximately,

$$\text{Bandwidth Ratio} \approx \frac{d \log n \prod_{i=1}^d \log n_i}{\log^2 m}.$$

Despite $n < m$ the value $\prod_{i=1}^d \log n_i$ can grow larger than $\log m$. For all tested datasets, the ratio of these two quantities never exceeds 26. The ratio is highest for real 3 dimensional data. We're not aware of any mechanism to reduce the bandwidth of segment trees while retaining accuracy; there are queries that require accessing the maximum number of nodes (see Section 3.1.1).

Branching-factor tradeoff. One can generalize the segment tree to branching factor r . As shown in Table 6, this reduces the height from $\sum_{i=1}^d \log n_i$ to $\sum_{i=1}^d \log_r n_i$, reducing rounds. This reduces the logarithmic factors in the worst-case access bound, however $(2r)^d$ replaces 2^d on the number of accesses (see discussion after Theorem 2). The value of r with the smallest bandwidth is e , but the relative quality of $r = 2$ and $r = 3$ depends on the values of n_i .

Evaluation: We present a prototype implementation of our approach testing on 1, 2 and 3 dimensional datasets with datasets as large as $5M$ records and segment trees as large as $40M$ records. For the largest dataset our parallel search time is 1.2s for a server with 32 cores, with a height of at most 27 yielding 28 rounds of communication yielding a total of 4s for a 100ms RTT. Additionally we find:

1. The number of nodes in the segment tree is substantially smaller than $2^d n$. The size of the actual segment tree is smaller by as much as a factor of 100 (depending on how "skewed" the dataset is).
2. Our storage savings scales with the inverse of the density of the dataset, n/m . For real data, we observe densities as low as 10^{-8} . We also test on synthetic datasets with density as small as 10^{-12} .

We implement a comparable oblivious map to benchmark prior work. We use Demertzis et al.'s prefix sum approach as a benchmark, it is the most storage efficient solution based on a map, Falzon et al.'s improvements can be viewed as supporting more functions [24]. Build time is slower by up to 10^9 seconds with a increase in storage of up to 10^5 . This disparity increases further for schemes with more storage.

1.3 Further Related Work

Espiritu et al. [21, 22] introduce a technique for dealing with sparse data. For a parameter $0 < \alpha < 1$ one creates a multimap from each subrange of size $m^{1-\alpha} := \prod_{i=1}^d m_i^{1-\alpha}$ that lists all of the existing values in

that range (and their associated aggregated value). This allows one to find the “next” existing item in the dataset.⁵ One has to store this multimap. Costs depend on the maximum number of items that can be associated with keyword, denoted as ρ . There is no natural way to bound the maximum number of items in each subrange and ρ directly impacts the efficiency of current oblivious multimaps. Espiritu et al. approach is best for data that has a small number of values in **every** hyper-rectangle. We show the number of stored items for this bucketed approach, denoted as EMT, in Table 3. In this comparison, we assume that each bucket is padded to length ρ as is common in oblivious multimaps. The number of stored items is almost always smaller than the prefix array but still approaches 10^{10} on 2 and 3 dimensional datasets. We don’t list ρ but it has a large impact on performance, for books 1D $\rho = 4$ and storage is 10^4 but it is dramatically worse for 2d and 3d datasets where the maximum number of items in a bucket ranges from $\rho = 200$ to $\rho = 30000$.

Kushnir, Moshkovich and Shaul [47] presented a construction called *copy and recurse* that uses fully homomorphic encryption (FHE). Copy and recurse computes the intersection of the queried range with the dataset, stored in a partition tree [54]. The tree is traversed under FHE. The subtrees needed for traversal are copied and search is repeated until search reaches the leaf of the tree where the result can be computed. The computational complexity of *copy and recurse* for general ranges is $t \cdot N^{1-\frac{1}{d}+\epsilon} + N^{1+\epsilon}$ where $\epsilon > 0$ is an arbitrary small value and t is the time complexity of comparing a node to a range. Recent estimates place $t \approx 25$ seconds, though several hundred comparisons can be executed simultaneously using SIMD instructions [68].

Kushnir, Moshkovich and Shaul did not present a scheme to compute an aggregated value using *Copy and Recurse*, but one can store the relevant range values and compute the aggregate function on them. SUM and MEAN have simple homomorphic computations. However, MIN, MAX, MEDIAN require further layers of comparison operations or sorting, this is an active area of research [13, 15]. STDDEV requires either absolute values or integer multiplication either of which is complex. We are not aware of any STDDEV implementations under FHE. This scheme is denoted as C&R in Table 3, it presents reasonable storage at the cost of inefficient search.

Our approach is appropriate for small d as many aspects of our complexity scale multiplicatively with the number of dimensions. This makes ours and prior approaches appropriate for scientific data where each value is numeric, but inappropriate for searching over high dimensional feature vectors. See related work on secure approximate nearest neighbor search; for example, compass [73] introduces a graph-based semantic search scheme from the Hierarchical Navigable Small World (HNSW) index [51].

Organization The rest of the work is organized as follows: Section 2 introduces preliminaries and notation including segment trees, Section 3 introduces the oblivious segment construction and efficiency analysis, Section 4 describes our implementation, datasets and experimental results, and Section 5 concludes.

2 Preliminaries

Let $\lambda \in \mathbb{N}$ be a security parameter. Throughout, all algorithms are considered collections of algorithms indexed by security parameter λ . However, λ is often omitted from the notation for simplicity. When we describe an interactive protocol between a client, $\mathcal{C}$, and server, $\mathcal{S}$, we use

$$\begin{pmatrix} \mathcal{C}_{\text{out}} \\ \mathcal{S}_{\text{out}} \end{pmatrix} \leftarrow \text{Prot} \begin{pmatrix} \mathcal{C}_{\text{in}} \\ \mathcal{S}_{\text{in}} \end{pmatrix},$$

to mean the $\mathcal{C}$ and $\mathcal{S}$ run the protocol Prot and that the $\mathcal{C}$ provides $\mathcal{C}_{\text{in}}$ and input and receives $\mathcal{C}_{\text{out}}$ while the $\mathcal{S}$ provides $\mathcal{S}_{\text{in}}$ and input and receives $\mathcal{S}_{\text{out}}$.

For integers a, b let $[a, b] = \{x \in \mathbb{Z} | a \leq x \leq b\}$ and let $[b]$ be shorthand for $[1, b]$. We consider a database $\mathcal{DB}$ where each tuple $\mathcal{DB}_i$ consists of d dimensions and an associated record. Everything in this work naturally extends to associating multiple values in the record, for simplicity we consider a single

⁵In the 1-dimensional case, Espiritu et al. show how to directly embed the aggregated value into this multimap. This technique does not naturally extend to multiple dimensions, but for the purposes of comparison we just measure the size of the multimap as though the technique works.

aggregatable value. For $1 \leq i \leq d$, let m_i denote the maximum value of the i th dimension which ranges in $[m_i]$. Let n_i be the number of unique existing values within the $[m_i]$ range, note that $1 \leq n_i \leq m_i$. A database $\mathcal{DB} = (\mathcal{DB}_1, \dots, \mathcal{DB}_N)$ is an *ordered* collection of tuples where each

$$\mathcal{DB}_j = (x_{j,1}, \dots, x_{j,d}, r_j) \in \left(\prod_{i=1}^d [m_i] \right) \times \mathcal{R}.$$

A query is $\vec{q} = (a_1, b_1), \dots, (a_d, b_d)$ where $1 \leq a_i \leq b_i \leq m_i$. Define

$$\mathcal{DB}_{\vec{q}} = (\mathcal{DB}_j | \forall 1 \leq i \leq d, a_i \leq x_{j,i} \leq b_i).$$

For any $\mathcal{DB}_{\vec{q}} \subseteq \mathcal{DB}$ we use $f(\mathcal{DB}_{\vec{q}})$ to represent the associative aggregation function f applied over the records r in $\mathcal{DB}_{\vec{q}}$. Associative aggregation functions include **SUM**, **MEAN**, **MAX**, **MIN**, **COUNT**, **STDDEV**. We note that for **MEAN** and **STDDEV** to be associative one must also have the value of **COUNT** for each subrange.

We consider databases organized as arrays A . For a single dimension, any comparison operator suffices to convert $\mathcal{DB}$ into an ordered array. After introducing segment trees [5] we discuss how to handle multidimensional data.

Definition 1 (1D Segment Tree). *A Segment Tree $\mathbf{ST}$, built on an array A , is a complete binary tree. Each intermediate node s_i corresponds to a segment from i_{start} to i_{end} of the array and stores the result of a function f applied over that segment:*

$$f([A[i_{start}], \dots, A[i_{end}]]).$$

The left and right children of node n_i , denoted as n_l and n_r , are defined over the ranges $[i_{start}, \lfloor \frac{i_{start} + i_{end}}{2} \rfloor]$ and $[\lfloor \frac{i_{start} + i_{end}}{2} \rfloor + 1, i_{end}]$, respectively. This process continues until the leaf nodes are reached. A leaf node n_l is associated with a single element $A[l]$ from the array and holds the value of $f([A[l]])$ computed over that specific element.

Figure 1 shows a 1D segment tree that computes **SUM** over ranges. To compute the sum of elements within a range $[i, \dots, j]$, we begin at the root node, each node iteratively divide its segment into two equal size segments parts. This traversal continues until one finds segments that *are covered* by the query range. We then recursively sum the values of these segments and return the total sum. As we discuss in the next section, the overall height of a segment tree is $\log(n)$ and any query can be answered by aggregating a constant number of nodes at each level of the tree. Throughout this work we focus on static data, but segment trees (and our oblivious versions) are effective at updates on array segments [18, 40].

Multi-dimensional Segment Tree A multidimensional Segment Tree [39, 55] extends the one-dimensional Segment Tree to higher dimensions. The simplest case is a 2D Segment Tree, defined on a matrix or 2D array A of size $n_1 \times n_2$. The 2D Segment Tree $\mathbf{ST}$ is a complete binary tree whose nodes operate on segments along the first dimension, dividing the range $[1, n_1]$ until reaching leaf nodes. However, unlike the one-dimensional Segment Tree, each node in $\mathbf{ST}$ contains another segment tree, $\mathbf{ST}'$, representing segments along the second dimension. The root of $\mathbf{ST}'$ operates over the entire range $[1, n_2]$, and its children divide this range until leaf nodes are reached. A 2D Segment Tree is a *segment tree of segment trees*. They allow one to query any rectangular region of the 2D array A . We note that the actual size of the second dimensional segment tree depends on the number of (second-dimensional) distinct values in the (first-dimensional) range. Thus, the maximum size of the full tree is $\prod_{i=1}^d (2n_i - 1)$ but the actual tree may be much smaller (see Table 3). The construction is shown in Figure 2. Higher-dimensional Segment Trees recursively add more layers of segment trees.

Definition 2 (Oblivious Data Structure [70]). *A data structure $\mathbf{DS}$ is oblivious, if there exists a polynomial time simulator $\mathcal{S}$, such that for every polynomial-length sequence of data structure operations $\mathbf{op} = ((\mathbf{op}_1, \mathbf{arg}_1), \dots, (\mathbf{op}_n, \mathbf{arg}_n))$,*

$$\mathbf{address}_{\mathbf{DS}}(\mathbf{op}) \stackrel{d}{=} \mathcal{S}(\mathcal{L}(\mathbf{op}))$$

where $\mathbf{address}_{\mathbf{DS}}(\mathbf{op})$ is the physical addresses generated by the oblivious data structure during a sequence of operations $\mathbf{op}$, and $\mathcal{L}(\mathbf{op}, \mathbf{DS})$ is the leakage function.

distinct: n	i:	1	2	3	4				
index: m	A	1	2	3	4	5	6	7	8
Values	B	5	-	4	-	8	-	-	2

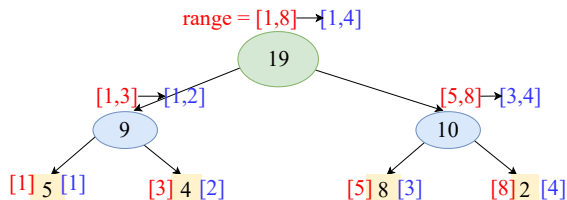


Figure 1: 1D-segment tree for a sample sparse data. A segment tree is constructed over the distinct indices, with each internal node storing the aggregate sum of its subtree (using $f = \text{SUM}$ in this example). For a range-sum query over $[5, 8]$, the tree directly returns the node whose interval matches this range, yielding the value 10.

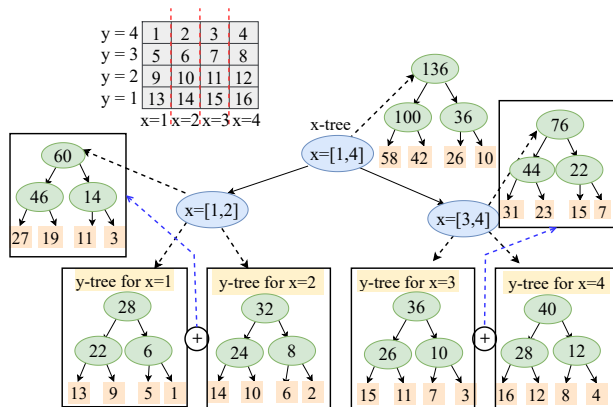


Figure 2: 2D-segment tree for a 4×4 example dataset. First, an x-tree is created for the range $x[1,4]$, containing the existing elements in the range. For each $x=i$ within $x[1,4]$, a corresponding y-tree is constructed for the strips $x=1, \dots, x=4$. Next, each leaf node of the x-tree is linked to its associated y-tree. Finally, the y-trees are merged by summing the values of corresponding nodes, continuing up to the root of the x-tree.

Remarks We don't formally define the correctness of an oblivious data structure but it is clear from context. All schemes considered in this work are correct (not approximate).

PathORAM ORAM is an oblivious data structure where operation `op` can be either reading from an address or writing to an address. The leakage is the number of addressable locations in the ORAM and the total number of read/write operations. PathORAM [67] is a well-known example where logical addresses are pseudorandomly mapped to physical addresses stored by a server. In addition, the server builds a tree over the leaves of its physical addresses. Reads and writes are both to paths in this tree, when a leaf is read from a logical address, all items in the path are placed in a stash and a new pseudorandom physical location is chosen for the logical location. The mapping from logical to physical location is a position map. In a standard *non-recursive* ORAM, the client stores the position map. In a *recursive* ORAM one recursively stores this position map in smaller ORAMs. Importantly, when one uses a recursive ORAM, one has to access the recursive ORAMs in sequence. Wang et al. [70] showed how to eliminate the position map for a rooted tree with bounded degree if the query functionality is path traversal of the tree. We generalize this notion slightly to consider traversing many paths. For a node `nd` one stores

$(id, range, Agg, count, CurPos, Add_\ell, Add_r, Add_n)$

Here, $\text{Add}_\ell/\text{Add}_r/\text{Add}_n$ store the current position of a left, right, the “root” of the next dimension respectively. Traversal works as follows:

1. One visits a node **nd** and uses the query to decide what children to visit,
2. For all children to be visited a new pseudorandom physical location is sampled and updated in **Add_ℓ** and/or **Add_r** as needed,
3. The nodes current address is updated to the location determined when its parent was visited in **CurPos**, and
4. The new value of **nd** is written in the location for **nd**.

Definition 3 (Oblivious Rooted Tree With Bounded Degree [70]). *An OTr is an oblivious data structure with the following two operations:*

$$\begin{pmatrix} sk \\ OM \end{pmatrix} \leftarrow \text{Setup} \begin{pmatrix} 1^\lambda, Tr \\ 1^\lambda \end{pmatrix}, \quad \begin{pmatrix} \vec{nd} \\ OM \end{pmatrix} \leftarrow \text{Query} \begin{pmatrix} Q, ndsLvl \\ OM \end{pmatrix}$$

with the following leakage functions

$$\begin{aligned} \mathcal{L}(\text{Setup}) &= s, d \\ \mathcal{L}(\text{Query}) &= ndsLvl \end{aligned}$$

Where s is the size of the tree and d is the number of dimensions. The length of $ndsLvl$ is the Height of the tree so this is leaked with every Query.

Query accesses nodes starting from the root, adaptively choosing which nodes to access at level i based on levels $1, \dots, i-1$ and queries exactly $ndsLvl_i$ nodes at level i , $\vec{nd}$ is a subset of the nodes accessed at level i . Not storing a position map is enabled by only accessing a node after its parent, and having parent nodes store the physical position of their children. The ability to update the position of a child before accessing the child is also crucial.

3 Oblivious Segment Trees

To secure the multi-dimensional segment tree we utilize oblivious RAM.⁶ We flatten the multidimensional segment tree and store the resulting array in a oblivious rooted tree with bounded degree using a single PathORAM array.

In the next subsection, we prove bounds on the number of segment tree accesses per level. These bounds are small enough that a modest server can process a meaningful fraction of requests for a single level in parallel. Across datasets, the highest value of accesses per level we see is 4 for 1-dimensional data, 100 for 2-dimensional data and 844 for 3-dimensional data (Table 10).

Our rounds are comparable to oblivious maps of the same size. As discussed above, our theoretical bounds on bandwidth are larger than prior work. Segment trees (and their oblivious versions) can only handle associative aggregation functions as the final value computed by the client aggregates multiple nodes found in the tree.

Statistics that cannot be aggregated We are not able to handle statistics that cannot be summarized efficiently such as **MODE** and **MEDIAN**. Prior work [21, 22] only consider approximate versions of **MODE** and **MEDIAN** only in a 1-Dimensional setting, their scheme cannot be extended to arbitrary dimensions. One can compute approximations of **MODE** and **MEAN** by traversing the segment tree and finding the largest node that wholly contains the query range. This is known as a range cover [20]. One can apply the same approach in a segment tree.

This section 1) gives a formal description of the oblivious segment tree and 2) show bounds on the height of the segment tree and number of accesses required per level. Our bound on the number of accesses is logarithmic in the product of $\prod_{i=1}^d \log n_i$ where n_i is the number of distinct values in each dimension. Such a bound is crucial for efficient worst-case traversal. We show a lower bound on the height of a quad tree under reasonable assumptions on the data. (Recall that worst case accesses for a quad tree is linear in N so it does not represent a viable solution.)

3.0.1 Creating an efficient oblivious segment tree

As stated in Definition 3, it suffices to design **Setup** and provide a bound $ndsLvl$ on the number of nodes that should be accessed at each level of traversal. Let **ST** be a segment tree.

⁶Recently, Shirotake and Shimizu [66] applied a 2-out-of-2 secret-sharing scheme on the segment tree to secure a range search but their work is in a different security model with little overlap in cryptographic techniques.

We bound the number of nodes that can be accessed in each tree level in the worst-case scenario, and in each level of the tree we access this number of nodes (the true number of accesses can be smaller). The main **Setup** and **Query** are shown in Algorithm 1. These algorithms describe how to use the following low level PathORAM algorithms to build **ApplyO** and **Query**:

1. $sk \leftarrow \text{Gen}(\perp)$ create an initial secret key.
2. $add \leftarrow \text{Malloc}(sk)$ create a new address for writing. The above two operations are local to the client in contrast to the other two operations which are interactive.
3. $\begin{pmatrix} \perp \\ OM' \end{pmatrix} \leftarrow \text{Write} \begin{pmatrix} add, nd, sk \\ OM \end{pmatrix}$ write a node to an address, addresses are written at most once.
4. $\begin{pmatrix} nd \\ OM' \end{pmatrix} \leftarrow \text{ReadAndEvict} \begin{pmatrix} add, sk \\ OM \end{pmatrix}$ read a node from an address, addresses are read at most once.

One frequently prepares an initial encrypted memory for efficiency reasons, we call this **Setup**, we focus on **Q** in Algorithm 1.

Query takes as input the number of accesses to be made at each level denoted by $ndsLvl$ in Algorithm 1. We elide the memory management of dummy nodes that are padded. One can simply keep the position of one dummy value at the client for each level of the oblivious segment tree. Also note that each level of the tree can be accessed in parallel, that is, all addressed are popped from the queue simultaneously. In Section 3.1, we compute and prove the upper bounds on the accessed nodes and the height of segment trees (and quad trees).

3.1 Efficiency and Security Analysis

We first analyze the upper bound of the number of segment tree nodes that can be returned and accessed per level and in total. Our bounds allow us to form $ndsLvl$ in a way that only depends on the height of the tree and the number of distinct values in each dimension. As mentioned above, leaking $ndsLvl$ leaks the height of the tree as well. We compare the height of segment tree and quad tree in Theorem 4, and show that using segment tree is more efficient as compared to using the quad tree in terms of height under a reasonable condition on data distribution. Finally in Theorem 5 we point out that our construction of the Segment tree satisfies the conditions of obliviousness due to the bounds on the degree of the tree.

It should be noted that in contrast to our approach that takes the existing data ranges, Falzon et al. [24] takes the domain range at each node and divide it by the branching factor to form its children. Thus our Segment-tree approach can be seen as Best/Uniform Range Cover (BRC/URC) discussed in [24, Theorem 2] on data points instead of the data domain where we always access the maximum number of nodes at each level.

3.1.1 Bounding Accesses of Segment Tree

Theorem 1. *The maximum number of returned and accessed nodes at each level of a 1 – dimensional segment tree is two and four respectively.*

Proof. Recall that $\vec{q}$ is a hyper rectangle so it is continuous in each dimension. Suppose that 3 nodes are returned at some level. The ranges covered by these nodes form a continuous range. Thus, it must be the case that for some j , nd_j, nd_{j+1}, nd_{j+2} are returned. It must be the case that either nd_j and nd_{j+1} share the same parent or nd_{j+1} and nd_{j+2} share the same parent.

Claim 1. *If two nodes at the same level are in the response set, then they do not have the same parent.*

Proof. Assume there are two nodes in the same level of the tree that satisfy a query, $\vec{q}$ and share the same parent. Let Add_ℓ, Add_r, nd denote the two nodes and their parent without loss of generality. Since $Add_\ell.range \subseteq \vec{q}$ and $Add_r.range \subseteq \vec{q}$ it follows that

$$nd.range = Add_\ell.range \cup Add_r.range \subseteq \vec{q}.$$

Algorithm 1: Algorithms for Oblivious Segment Tree

Setup^{c,s} $\left(\begin{smallmatrix} 1^\lambda, \text{Tr} \\ \perp \end{smallmatrix} \right)$

1. Initialize $\text{NdAdd} = \{\}$.
2. Set $\text{sk} \leftarrow \text{Gen}(1^\lambda)$.
3. Send $\text{OM} = \perp$ to $\mathcal{S}$.
4. For each $\text{nd} \in \text{Tr}$,
 - (a) Let $\text{NdAdd}[\text{nd}] = \text{Malloc}(\text{sk})$
5. For $\text{Level} = \text{Height}(\text{Tr})$ to 1:
 - (a) For $\text{nd} \in \text{Tr}_{\text{Level}}$, replace $\text{nd.Add}_\ell = \text{NdAdd}[\text{Add}_\ell]$, $\text{nd.Add}_r = \text{NdAdd}[\text{Add}_r]$, $\text{nd.next} = \text{NdAdd}[\text{next}]$.

$$\left(\begin{smallmatrix} \perp \\ \text{OM} \end{smallmatrix} \right) \leftarrow \text{O.Write} \left(\begin{smallmatrix} \text{NdAdd}[\text{nd}], \text{nd}, \text{sk} \\ \text{OM} \end{smallmatrix} \right)$$
 - (b) If $\text{Level} = 1$, set $\text{Root}_{\text{add}} = \text{NdAdd}[\text{nd}]$.
6. Server receives OM and client keeps $(\text{sk}, \text{Root}_{\text{add}})$.

Query^{c,s} $\left(\begin{smallmatrix} \vec{q}, (\text{sk}, \text{Root}_{\text{add}}), \text{ndsLvl}, \text{sk} \\ \text{OM} \end{smallmatrix} \right)$

1. Define $\text{queue} = []$, $\text{queue}_{\text{Next}} = []$, $\text{Res} = []$, $\text{Level} = 0$ and $\text{Acc} = 0$.
2. $\text{Root}_{\text{add}} \leftarrow \text{Malloc}(\text{sk})$. Set $\text{nd}_{\text{Level.add}} = \text{Root}_{\text{add}}$.
3. $\text{queue.push}(\text{nd}_{\text{Level.add}})$
4. Set $\text{nd}_{\text{Level.next}} \leftarrow \text{Malloc}(\text{sk})$. Set $\text{Root}_{\text{add}} = \text{nd}_{\text{Level.next}}$.
5. Pad/Truncate queue such that $|\text{queue}| = \text{ndsLvl}[0]$
6. While queue is not empty:
 - (a) $\text{add} \leftarrow \text{queue.pop}()$
 - (b)

$$\left(\begin{smallmatrix} \text{nd}_{\text{Level}} \\ \text{OM}' \end{smallmatrix} \right) \leftarrow \text{O.ReadAndEvict} \left(\begin{smallmatrix} \text{add}, \text{sk} \\ \text{OM} \end{smallmatrix} \right).$$
 - (c) Set $\text{nd}_{\text{Level.next}}$ from prior level.
 - (d) $\text{Acc}++$.
 - (e) If $\text{nd.range} \subseteq \vec{q}$:
 - i. If nd.isLeaf , $\text{Res.push}(\text{nd.Agg})$.
 - ii. Else $\text{queue}_{\text{Next.push}}(\text{nd.Add}_n)$. Set $\text{nd.Add}_n \leftarrow \text{Malloc}(\text{sk})$.
 - (f) Else if $\text{nd.range} \cap \vec{q} \neq \emptyset$
 - i. $\text{queue}_{\text{Next.push}}(\text{nd.Add}_\ell)$.
Set $\text{nd.Add}_\ell \leftarrow \text{O.Malloc}(\text{sk})$.
 - ii. $\text{queue}_{\text{Next.push}}(\text{nd.Add}_r)$.
Set $\text{nd.Add}_r \leftarrow \text{O.Malloc}(\text{sk})$.
 - (g) $\left(\begin{smallmatrix} \perp \\ \text{OM}' \end{smallmatrix} \right) \leftarrow \text{O.Write} \left(\begin{smallmatrix} \text{nd}_{\text{Level.next}}, \text{nd}_{\text{Level}}, \text{sk} \\ \text{OM} \end{smallmatrix} \right)$
 - (h) If $\text{queue} = []$ & $\text{Level} < |\text{ndsLvl}|$:
 - i. Set $\text{Acc} = 0$ and $\text{Level}++$.
 - ii. Pad/Truncate $\text{queue}_{\text{Next}}$ such that $|\text{queue}_{\text{Next}}| = \text{ndsLvl}[\text{Level}]$. (Padded values are fresh from Malloc .)
 - iii. Set $\text{queue} = \text{queue}_{\text{Next}}$ and $\text{queue}_{\text{Next}} = []$.
7. Return Res .

This is a contradiction and completes the proof of Claim 1 □

We now consider accessed nodes. First note that either both the children of a node are accessed or neither. For the sake of contradiction, suppose that at least 6 nodes are accessed at level i . For a node nd , let $\text{nd}_{\text{Parent}}$ be its parent. For nd to be accessed, it must be the case that $\vec{q} \cap \text{nd}_{\text{Parent}}.\text{range} \neq \emptyset$. Let $\text{nd}_\ell, \text{nd}_i, \text{nd}_r$ be the parents of the six nodes accessed at level i . The meaning of the subscripts will be clear

in the following argument. For these three nodes, since $\vec{q}$ is continuous, it must be the case that for one of these nodes, denoted as nd_ℓ , where $\text{nd}_\ell.\text{range} = [a_\ell, b_\ell]$, it is true that for all $x \in \vec{q}$, $x > a_\ell$. Similarly, for one of these nodes denoted as $\text{nd}_r = [a_r, b_r]$ it is true that for all $x \in \vec{q}$, $x < b_r$. Lastly, since nodes contain continuous ranges for $\text{nd}_i.\text{range} = [a_i, b_i]$ it must be the case that $b_\ell < a_i$ and $b_i < a_r$. Thus, one has that $\text{nd}_i.\text{range} \subseteq \vec{q}$ and nd_i 's children are not accessed. This is a contradiction and thus at most 4 nodes are accessed at any level. This completes the proof of Theorem 1. $\square$

Before moving on we note that the above theorem can be generalized to a segment tree with branching r instead of 2 with bounds of $2r - 2$ nodes being returned and $2r$ nodes accessed by the same argument: 1) if at least $2r - 1$ nodes are returned some parent node would have been returned, and 2) if at least $2r + 1$ nodes are accessed some nd_i parent node would have been returned.

Theorem 2. *The maximum total number of nodes in a 1D segment tree that can be returned is $2(\log n - 2)$, where $\log n$ is the height of the segment tree.*

Proof. For a tree of height $\log n$, we see the maximum number of nodes id returned when no nodes from level 1 or two is returned – this is because, 1) if the root of the tree is returned, the tree traversal is stopped and the number of returned node is one. 2) If one of the two nodes in second level is returned then the tree traversal in that subtree is stopped and the number of returned nodes is less than possible maximum. From Theorem 1 for a tree of height $\log n$, the maximum number of returned nodes is the maximum number of returned nodes per each level times the height of the tree minus the aforementioned levels which is $2 * (\log n - 2)$. $\square$

Theorem 3. *The maximum total number of returned and accessed nodes in a d – dimensional segment tree is*

$$\begin{aligned} \# \text{Returned} &\leq \sum_{i=1}^d \prod_{j=1}^i (2(\log n_i - 2)), \\ \# \text{Accessed} &\leq \sum_{i=1}^d 2 \prod_{j=1}^i (2(\log n_i - 2)). \end{aligned}$$

Proof. From Theorem 2, the maximum number of returned nodes in each 1D tree is $2(\log n - 2)$, we extend this to a multi-dimensional tree. A subtree is accessed in dimension $i + 1$ if and only if the $\vec{q}_i$ would return a node in dimension i (viewed as a 1-dimensional tree). The total maximum number of returned nodes is the sum of all the subtrees, which is $\sum_{i=1}^d \prod_{j=1}^i (2(\log n_i - 2))$. The maximum number of accessed nodes is multiplied by 2 to account for accessing in the “last” subtree for each dimension. $\square$

Tightness of Theorem 3 The above bounds are tight up to an additive constant. Consider a dense dataset where each dimension is over the same range $[1, m^*]$ where m^* is a power of 2. In this setting, every subtree has the same height. Then the query for $[2, m^* - 1]^d$ at each level starting from the third level, four nodes are accessed at each level: the middle two nodes are returned, while the query continues to traverse the children of the outer two nodes. This pattern repeats until the final level, where the middle two nodes are returned and the outer two nodes are discarded. This pattern occurs in every subtree that is traversed.

Accesses close to this worst case arise when a query hyper-rectangle repeatedly intersects partition boundaries across multiple levels of the segment tree in all dimensions. In contrast, queries aligned with partition boundaries, or queries that fully cover large subtrees, terminate earlier and require fewer accesses.

3.1.2 Bounding Height of Segment and Quad Trees

We now show that a quadtree [25] (and their generalizations to multiple dimensions) have height of at least as high as a segment tree under reasonable assumptions on the data distribution. This is true for all of our datasets (Table 3).

We start with a quick review of a quadtree. Each node of a quadtree simultaneously splits each dimension by the median of the current dimension. We use the terminology quad tree to refer to an arbitrary number of dimensions rather than oct tree or hyper oct tree. We also note that a “1D quad tree” is a segment tree.

Before introducing our theorem, we need an additional piece of notation. Recall we use n_i to refer to the number of distinct values for dimension i . We use the notation n_{2,x^*} to refer to the number of distinct values in dimension 2 that have the x -value of x^* . Note that $n_{2,x^*} \leq n_2$.

Theorem 4. *Let $\mathcal{DB}$ be a database over $[1, m_1] \times [1, m_2]$. Let $\mathcal{ST}$ be a segment tree with branching factor of 4 and let $\mathcal{QT}$ be a quadtree. When dimensions can be ordered in such a way that $n_1 \leq \max_{x^*} n_{2,x^*}$ for all x^* then*

$$\begin{aligned} \text{HeightQT}(\mathcal{DB}) &\geq \max_{x^*} (\log n_{2,x^*}) \\ \text{HeightST}(\mathcal{DB}) &\leq \frac{\log n_1 + \max_{x^*} (\log n_{2,x^*})}{2} \end{aligned}$$

We prove Theorem 4 through two lemmas establishing each of the above equations.

Lemma 1. *A longest path in a 2D-segment tree passes through an x -tree leaf. The height of all such paths is at most $\log n_1 + \max_{x^*} \log n_{2,x^*}$.*

Proof. We slightly extend the notation of n_{2,x^*} to refer to ranges using $n_{2,[a,b]}$ to refer to the number of distinct y values under a range $[a,b]$. For the sake of contradiction, assume a longest path does not pass through an x -tree leaf; so there is a path of a segment tree that passes through a y -tree (called $Y_{i,j}$) that is connected to an x -tree node $\text{nd}_{i,j}$ at intermediate level $i < \log m_1$ that is longer than all paths through leaf x nodes. In such a case, the height of $Y_{i,j}$ must be longer than the height of the y -tree connected to the x -tree's leaf node. We show this is not possible.

In a segment-tree with branching factor of 4, the intermediate node $\text{nd}_{i,j}$ at level i , has for children $\text{nd}_{i+1,1}, \text{nd}_{i+1,2}, \text{nd}_{i+1,3}, \text{nd}_{i+1,4}$. The number of distinct values in dimension $i+1$ for node $\text{nd}_{i,j}$ is maximized when each child has n_{i+1,x_j^*} have no values in common. In this case the maximum number of distinct values in next dimension for $\text{nd}_{i,j}$ is $\sum_{j=1}^4 n_{2,x_j^*} \leq 4 * \max_{x_j^*} n_{2,x_j^*}$. This concludes the maximum height for the Y -tree rooted at node $\text{nd}_{i,j}$ is at most $1 + \max_{x^*} \log_4 n_{2,x^*}$. Thus, the height of the Y -tree rooted at node $\text{nd}_{i,j}$ is at most as long as the height of the Y -tree of one of its children. The statement argues by applying the argument all the way to an X -leaf node. $\square$

Lemma 2. *The longest path in a 2D-quad tree has length at least $\max_{x^*} (\log n_{2,x^*})$.*

Proof. Consider one x^* that maximizes $\log n_{2,x^*}$. Consider a node nd_i at height i that contains x^* , and let n_{2,i,x^*} be the number of distinct y values associated with x^* at this node. at most two children of nd_i will contain x^* so at least one of these children nodes nd_{i+1} will have $n_{2,i+1,x^*} \geq n_{2,i,x^*}/2$. By repeating this argument, there are at least $\log n_{2,x^*}$ total levels in the $\mathcal{QT}$. $\square$

The above arguments extend directly to d -dimensional quad trees in comparison with a segment tree with branching factor 2^d .

Theorem 5. *The construction described in Algorithm 1 is a oblivious data structure that implements the segment tree functionality.*

Proof Sketch. First note that the tree is of bounded degree at most 3. Our Oblivious Segment tree construction satisfies the properties of a rooted bounded degree tree described in Definition 3. Wang et al. [70] proved that the procedures in Algorithm 1 makes the data structure oblivious. $\square$

4 Implementation and Experiments

We implemented the oblivious segment tree structure with PathORAM as the underlying non-recursive ORAM. We have developed implementations for 1D, 2D, and 3D segment trees. The 1D implementation follows the construction methodology detailed in Section 3. In addition, we implemented an unprotected quad tree for comparing height and an oblivious map for comparing storage and computational requirements with prior work. For the 2D segment tree, we first construct an x -tree. For each node in the x -tree, we

associate a y-tree, which is a 1D segment tree built using the y-strip data corresponding to that node. Initially, the leaves of the x-tree are linked to individual y-trees. These y-trees are then recursively merged using a depth-first search (DFS) algorithm to form a unified y-tree, which remains a 1D segment tree. The two binary trees with differing structures, which is handled by a DFS-based merge function. The process continues until the root of the x-tree is associated with the merged y-tree of its children.

A similar approach is employed for the 3D segment tree, with the key distinction being that the merging process operates on 2D segment trees instead of 1D trees. The DFS-based algorithm used for merging is adapted accordingly to handle the additional complexity.

Oblivious Segment Tree Our implementation of oblivious segment tree uses PyORAM module as the main cryptographic primitive. PyORAM is an implementation of PathORAM [67] which is built from PRFs. In our implementation we make accesses at each level sequentially but record the maximum time of any access at a level to compute parallel time (which is the sum of the max at each level for each query). For each dataset we compute the maximum number of accesses per level across 1000 random queries and record this to understand the tightness of our analytic computation of maximum number of accesses from the previous section.

4.1 Experiments

Experimental Setup All evaluations and benchmarks are performed on a server with AMD Ryzen Threadripper PRO 7995WX CPU with 96 physical cores (192 threads) and 768 GB of RAM, running Ubuntu 22.04. The ORAM was stored on disk and loaded a single time with each query and written back at the end of each query. Our time measurements include this loading time. Our parallel time measures the maximum time to perform an ORAM access at each level of the oblivious segment tree (averaged across queries). Importantly, the parallel size does not consider the disk read and write. The code for our experiments is available at the github repository.

Datasets We used seven real datasets from prior work [20, 22, 24] as well as synthetic datasets (Table 7). For datasets from prior work, we follow their methodology.

- *Gowalla* [22] social networking location/time checkins.
- *Amazon 1D* [22] 51,311,621 book reviews.
- *Spitz 2D* [24] phone location data from Malte Spitz.
- *gowalla 2D* We sampled 50K, and 100K latitude and longitude values from the Gowalla 4D, and normalized the values in ranges represented in the Table 3.
- *Cali 2D* Locations of road network intersections in California used in [24].
- *nh 3D* [24] 4096 elevation points on domain $[64] \times [64] \times [64]$ from the White Mountains of New Hampshire.
- *gowalla 3D* We sampled a 23K dataset of latitude, longitude, and location-ids from gowalla 4D and normalized.
- *synthetic 2D-2048-d* 1 million randomly 2D points in $[2^{11}] \times [2^{11}]$ with 2^{11} unique values in each dimension.
- *synthetic 2D-2048-sp* 1 million randomly sampled 2D points in $[1M] \times [1M]$ with 2^{11} unique values in each dimension.
- *synthetic 2D-1024* 1 million randomly sampled 2D points in $[1M] \times [1M]$ with 2^{10} unique values in each dimension.
- *synthetic 3D-128* 1 million randomly sampled 3D points in the range $[1M] \times [1M] \times [1M]$ with 2^7 unique values in each dimension.
- *synthetic 3D-256* 1 million randomly sampled 3D points in the range $[1M] \times [1M] \times [1M]$ with 2^8 unique values in each dimension.

dataset	N	n	m	Density
books 1D	7836	7836	[704.9K]	-2.0
gowalla 1D	5×10^6	5×10^6	[54M]	-1.0
Spitz 2D	1480	662, 524	[967], [1024]	-2.8
cali 2D	17.7K	1015, 961	[1024], [961]	-1.7
gowalla 2D	50K	2127, 2048	[93K], [74.2K]	-6.1
gowalla 2D	103K	6438, 4500	[119K], [743K]	-5.9
synthetic 2D	1M	2048 ²	[2048], [2048]	-0.6
synthetic 2D	1M	2048 ²	[998K], [1M]	-6.0
synthetic 2D	1M	1024 ²	[999K], [998K]	-6.0
nh 3D	4096	64 ³	[64], [64], [64]	-1.8
gowalla 3D	23K	33, 97, 101	[100], [100], [100]	-1.6
synthetic 3D	1M	128 ³	[974K], [973K], [986K]	-12.0
synthetic 3D	1M	256 ³	[979K], [981K], [991K]	-12.0

Table 7: Summary of Datasets. Density is presented in logarithmic (base 10) scale.

Queries We generated random hyper-rectangular range queries within the domain range to retrieve aggregated values over the specified ranges. 1000 random queries were generated for each dataset.

4.2 Results Analysis

4.2.1 Storage Efficiency

Table 3 is in the Introduction and presents a comparative analysis of the storage requirements of the proposed *Oblivious Segment Tree* approach against prior work.

- *Map based storage proportion to m* [38] The storage requirement is the grid size, which is the $\Pi_{i=1}^d m_i$, where m_i is the domain range along the dimension i [65].
- *Min Aggregation* [20] The storage size is measured as $\Pi_{i=1}^d m_i \cdot \log m_i$.
- *EMT* [22] supports associative aggregation. The size of **MM** is measured according to their proposed bucketing method, in which stored items are padded to the maximum number of items associated with any keyword.
- *C&R* [47] This is the *copy & recurse* approach based on FHE, and we measured the size of the partition tree.

The segment tree method demonstrates **substantial storage savings** across most datasets. There are three datasets where the oblivious segment tree is larger: the 2D synthetic dataset, the 3d nh dataset, and the 3D gowalla dataset. Our size is never more than an order of magnitude larger.

However, the segment tree often has savings of multiple orders of magnitude, representing an implementable solution for many datasets where prior solutions storage cannot be implemented. This is confirmed in our cryptographic experiments shown in Table 9 where we compare the cryptographic storage of the two approaches. We note that we were not able to complete all experiments that required storing at least m items, so some numbers are interpolated in this table (see caption for details).

This saving is even more pronounced for the Min scheme [20], achieving an improvement by a factor of up to 10^{14} . Recall that all associative aggregate functions can be handled by our segment tree structure. Also, in the bucketing approach of EMT [22], the overall storage exceeds the segment tree size, and in higher dimensions, the storage overhead can be up to 10^5 times greater. In addition, for the *copy & recurse* [47] we never exceed their storage by more than a factor of 8. However, our search is much faster than theirs as discussed in the Introduction.

Lastly, we compute the bandwidth of our scheme in comparison to an oblivious map implementation that stores m items (Table 9). As discussed in the Introduction, bandwidth is one area where our performance is worse than prior constructions. Our bandwidth (in relation to Demertzis et al.) degrade with more

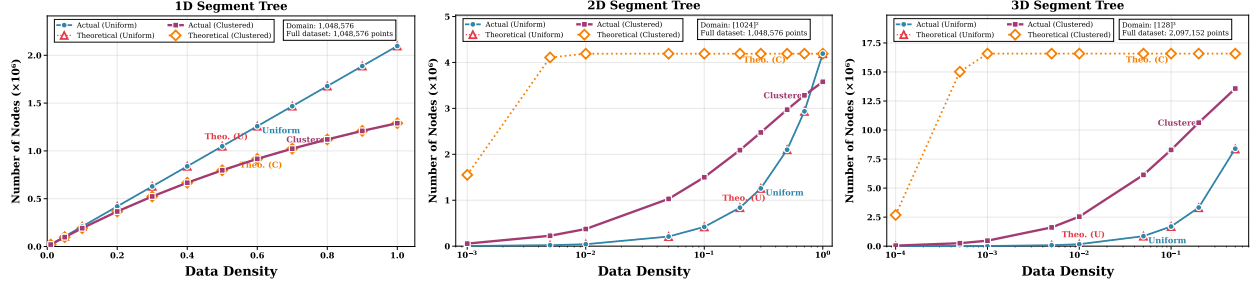


Figure 3: Storage versus data density for 1D, 2D, and 3D segment trees under uniform and clustered synthetic datasets.

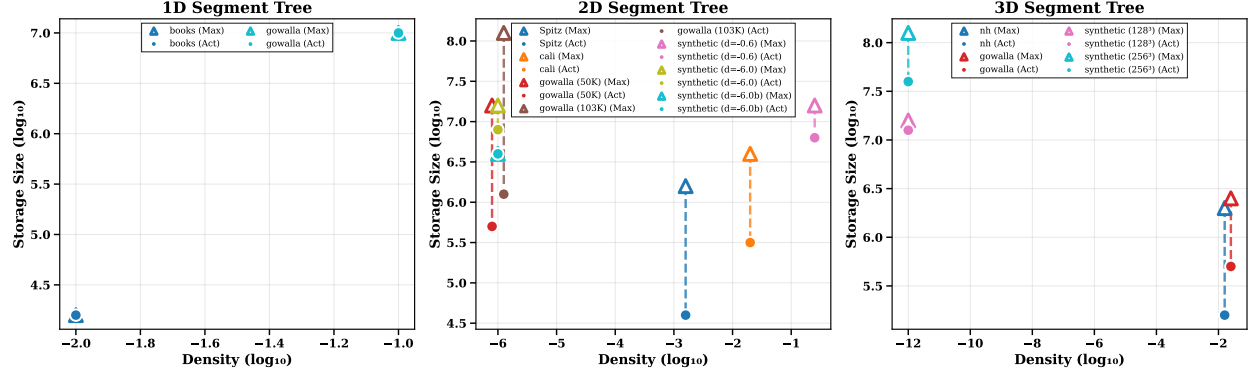


Figure 4: Storage savings versus density across the datasets.

dimensions. Overall, as the dimensionality of datasets increases, the storage requirements of prefix-sum, Min aggregation, and EMT grow rapidly. The segment tree approach shows a more scalable storage trend. As datasets become sparser, this trend is enhanced even more, with our storage savings often approaching the dataset’s density.

Storage Efficiency vs. Density: To further isolate the effect of sparsity, we evaluate storage as a function of data density using controlled synthetic datasets in 1D, 2D, and 3D under both uniform and clustered distributions, where points are concentrated in localized regions. Figure 3 shows that the storage cost of our segment-tree index is governed primarily by the number of distinct populated coordinates rather than the full domain size, so storage remains close to the realized data support and improves as density decreases. In 1D, storage grows roughly linearly with density for both uniform and clustered data, and the empirical curves closely follow the theoretical predictions. In higher dimensions, the gap becomes much more pronounced: under uniform data, storage decreases rapidly with density, while under clustered data the realized tree remains substantially smaller than the worst-case clustered bound, showing that practical structure sharing is much better than the pessimistic upper bound suggests. Most importantly, the advantage strengthens with dimensionality—especially in 2D and 3D—where low-density datasets lead to orders-of-magnitude savings. This trend is consistent with our broader experimental finding that storage savings scale with inverse density. The storage savings versus data density across the datasets used in our cryptographic experiments are illustrated in Figure 4. Storage savings depend on both density and data distribution. Across the real-world datasets, stronger clustering yields greater storage savings. In contrast, although the synthetic datasets are highly sparse, their more uniform distribution results in smaller savings.

4.2.2 Cryptographic Efficiency

Table 9 evaluates the performance of the oblivious segment tree on 1000 random aggregation queries.

dataset	H	#Returned	# A	$O.\#A$
Books 1D	14	11	32	49
gowalla 1D	24	20.0	59	89
Spitz 2D	22	9	54	1083
cali 2D	22	20	91	1083
gowalla 2D	25	6	46	1535
gowalla 2D	28	10	67	1947
synthetic 2D-2048-d	24	36	140	1339
synthetic 2D-2048-sp	24	65	231	1339
synthetic 2D-1024	22	53	187	1083
nh 3D	21	9	49	4115
gowalla-3D	23	9	38	5699
synthetic 3D-128	24	78	273	7055
synthetic 3D-256	27	133	475	11315

Table 8: Query Information. **Returned** is the average number of returned nodes from the ORAM structure, and **A** is the average number of required accesses in the plain text segment tree. $O.\#A$ is the number of accesses in the oblivious version.

Storage The largest required storage was 71GB, representing an expansion factor of roughly 1750 over the number of nodes in plain-text segment tree and roughly 70000 over the number of records in the database. Note that each node in the segment tree is represented by $2 + 2^d$ integers of the node information and up to 3 ORAM addresses (left and right child and next dimension child). As stated above we also implement an oblivious map implementation of Demertzis prefix sum scheme. Due to its larger storage requirements, our system exhausted RAM for some of the larger datasets, forcing us to interpolate numbers for the largest datasets. Our storage is often decreased by multiple orders of magnitude. This improvement is even larger for Demertzis min scheme.

Time We measure three timing quantities shown in Table 9. The time to build the oblivious is on the order of days for our largest datasets and is performed only once.

The number of rounds for the protocol is the number of levels traversed in the segment tree plus one. For the multidimensional segment tree, we used a binary tree per dimension, where each node stores the aggregated value, range, position tags of the right child, left child, and the root of the next dimension. This accounts for at most $(10 \cdot 32 + 3 \cdot 64) / 8 = 64$ bytes of storage out of the 256-byte ORAM block size. To reduce the number of rounds—we can use trees with higher branching factors. For example, one could construct a 27-ary tree (the remaining 192 bytes of storage yields 24 additional 8 byte ORAM addresses). This would reduce the tree depth and the required number of round trips by a factor of $\log_2 27 \approx 4.8$. As mentioned above, the results of the previous subsection can be extended to larger branching factors.

We measure the overhead of making the maximum number of oblivious accesses at each level (Table 8). This represents the additional work being performed in comparison with a plaintext query. As an example, for the Books 1D dataset, the average query accesses 32 nodes while the oblivious requires always accessing 49 nodes ($O.\#A$), for an access overhead of $\approx 50\%$. This overhead increases as the number of dimensions increases with a multiplicative overhead being at most 1.50 on the 1D datasets, 30 on the 2D datasets, and climbing to at most 150 on the 3D datasets.

This is shown visually in Figure ???. The number of accessed nodes goes from a standard deviation of 1 on the 1-dimensional dataset to almost 40 on the largest 3-dimensional datasets. So both the average and standard deviation climb for accessed nodes as the dimension increases.

The parallel time (Table 9) measures the sum of the maximum time to make each access at a level of the segment tree (averaged across queries). The sequential time represents an upper bound on computation time while parallel time represents a lower bound on computation time. We note that for most datasets, even a fast network with 100ms round trip time, the network delay will be ≈ 2.8 seconds. Thus, with a moderate amount of parallel computation, network delay represents the dominant cost. If a server could process the 850 accesses at the widest point of the 3D segment tree in groups of 32 the parallel time is ≈ 1.2 s for a total of 4s.

dataset	Oblivious Segment Tree						Demertzis Prefix Array			
	Setup	Rd.	Search	Par.	Size	BW	Setup	Search	Size	BW
Books 1D	15	15	0.05	0.01	17.0MB	171.0KB	830	.0018	1.5GB	190KB
gowalla 1D	14819	25	2.1	0.02	17.8GB	507.0KB	64K	0.0023	117GB	253KB
Spitz 2D	49	23	0.55	0.02	69MB	3.4MB	1.2K	0.0018	2.2GB	408KB
cali 2D	413	23	0.73	0.02	560MB	4.2MB	1.2K	0.0018	2.1GB	408KB
gowalla 2D	756	26	1.1	0.024	1.1GB	5.7MB	8M	0.0030	15TB	1.1MB
gowalla 2D	2220	29	1.7	0.029	2.2GB	8.1MB	104M	0.0033	190TB	1.4MB
synthetic 2D-2048-d	9811	25	2.4	0.03	8.9GB	5.7MB	4.9K	0.0020	9.1GB	497KB
synthetic 2D-2048-sp	15684	25	3.2	0.027	18GB	5.7MB	1.2B	0.0036	2.2PB	1.6MB
synthetic 2D-1024	6127	23	1.7	0.021	4.4GB	4.3MB	1.2B	0.0036	2.2PB	1.7MB
nh 3D	280	22	2.24	0.022	278MB	17MB	308	0.0016	569MB	665KB
gowalla-3D	706	24	3.45	0.027	1.1GB	18MB	1.2K	0.0018	2.2GB	816KB
synthetic 3D-128	23437	25	8.37	0.035	18GB	31MB	10^{15}	0.0054	2.0ZB	7.2MB
synthetic 3D-256	97118	28	21.02	0.046	71GB	54MB	10^{15}	0.0054	2.1ZB	10.0MB

Table 9: Measuring the ORAM performance over 1000 sample aggregation queries. Rd. is the average access depth plus one, and E-stop.Rd. is the same value with the early stop optimization condition, Par. is the sum of the (average of) the maximum ORAM retrieval at each depth. For the Demertzis Cryptography columns, we benchmark an tree-based oblivious map [70] on sizes $\ell = 2^5, 2^5, \dots, 2^{21}$. We perform a linear fit on storage and build of ℓ and search on $\log \ell$. The R^2 for these three fits is 1.00, .999, and .98 respectively. Setup, search, and Par. are setup time, search time, and parallel search time in seconds. This scheme which only supports SUM is chosen as a benchmark as it is more efficient than Falzon et al. [24] which supports all associative aggregation functions.

5 Conclusion

We introduce oblivious multidimensional segment trees that support associative aggregation queries over hyper-rectangular domain ranges. Our storage overhead is orders of magnitude lower than prior work, making secure outsourcing feasible for datasets that were previously too expensive. Our approach is built on careful use of oblivious data structures instantiated using ORAM.

The only information leaked is limited to the size and number of nodes in the segment tree and setup, and the number of accessed nodes and height at search. We present bounds for accessed nodes that depend only on the number of distinct values n_i in each dimension.

Cryptographic experiments illustrate the practicality of our scheme, as evidenced by the encrypted structure storage being 71GB for the 3D dataset with a segment tree size of 40M.

Acknowledgements

The authors thank the anonymous reviewers for their help in improving the manuscript. The work of all authors was supported by NSF grant #2232813. M.R. and B.F. were additionally supported by NSF grant #2141033.

References

- [1] J. M. Abowd. The us census bureau adopts differential privacy. In *KDD*, pages 2867–2867, 2018.
- [2] A. Appan, D. Heath, and L. Ren. Oblivious single access machines-a new model for oblivious computation. In *CCS*, pages 3080–3094, 2024.
- [3] G. Asharov, I. Komargodski, W.-K. Lin, E. Peserico, and E. Shi. Optimal oblivious parallel ram. In *SODA*, pages 2459–2521. SIAM, 2022.

- [4] M. A. Bender, M. Farach-Colton, G. Pemmasani, S. Skiena, and P. Sumazin. Lowest common ancestors in trees and directed acyclic graphs. *Journal of Algorithms*, 57(2):75–94, 2005.
- [5] J. L. Bentley. Algorithms for klee’s rectangle problems. Technical report, Technical Report, Computer, 1977.
- [6] A. Boldyreva and T. Tang. Encrypted multi-map that hides query, access, and volume patterns. In *International Conference on Security and Cryptography for Networks*, pages 230–251. Springer, 2024.
- [7] D. Boneh, E. Boyle, H. Corrigan-Gibbs, N. Gilboa, and Y. Ishai. Lightweight techniques for private heavy hitters. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 762–776. IEEE, 2021.
- [8] C. Bösch, P. Hartel, W. Jonker, and A. Peter. A survey of provably secure searchable encryption. *ACM Computing Surveys (CSUR)*, 47(2):1–51, 2014.
- [9] E. Boyle, N. Gilboa, and Y. Ishai. Function secret sharing: Improvements and extensions. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, pages 1292–1303, 2016.
- [10] E. Boyle, N. Gilboa, Y. Ishai, and V. I. Kolobov. Programmable distributed point functions. In *Annual International Cryptology Conference*, pages 121–151. Springer, 2022.
- [11] X. Cao, W. Feng, J. Liu, J. Zhou, W. Fang, L. Wang, Q. Xu, C. Yang, and K. Ren. Towards practical oblivious map. *VLDB*, 2025.
- [12] D. Cash, P. Grubbs, J. Perry, and T. Ristenpart. Leakage-abuse attacks against searchable encryption. In *Proceedings of the 22nd ACM SIGSAC conference on computer and communications security*, pages 668–679, 2015.
- [13] G. S. Cetin, E. Savaş, and B. Sunar. Homomorphic sorting with better scalability. *IEEE Transactions on Parallel and Distributed Systems*, 32(4):760–771, 2020.
- [14] Z. Chang, D. Xie, F. Li, J. M. Phillips, and R. Balasubramonian. Efficient oblivious query processing for range and knn queries. *IEEE Transactions on Knowledge and Data Engineering*, 34(12):5741–5754, 2021.
- [15] A. Chatterjee and I. Sengupta. Sorting of fully homomorphic encrypted cloud data: Can partitioning be effective? *IEEE Transactions on Services Computing*, 13(3):545–558, 2017.
- [16] H. Davis, C. Patton, M. Rosulek, and P. Schoppmann. Verifiable distributed aggregation functions. *Cryptology ePrint Archive*, 2023.
- [17] M. De Berg. *Computational geometry: algorithms and applications*. Springer Science & Business Media, 2000.
- [18] M. de Berg, M. van Kreveld, M. Overmars, and O. Schwarzkopf. Computational geometry. In *Computational Geometry*. Springer, Berlin, Heidelberg, 1997.
- [19] I. Demertzis, D. Papadopoulos, C. Papamanthou, and S. Shintre. SEAL: Attack mitigation for encrypted databases via adjustable leakage. In *29th USENIX Security Symposium (USENIX Security 20)*, pages 2433–2450, 2020.
- [20] I. Demertzis, S. Papadopoulos, O. Papapetrou, A. Deligiannakis, M. Garofalakis, and C. Papamanthou. Practical private range search in depth. *ACM Trans. Database Syst.*, 43(1), mar 2018.
- [21] Z. Espiritu. Time-and space-efficient aggregate range queries on encrypted databases. *Brown University*, 2022.
- [22] Z. Espiritu, E. A. Markatou, and R. Tamassia. Time-and space-efficient aggregate range queries over encrypted databases. *PoPETS*, 2022(4), 2022.

- [23] F. Falzon, E. A. Markatou, D. Cash, A. Rivkin, J. Stern, and R. Tamassia. Full database reconstruction in two dimensions. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pages 443–460, 2020.
- [24] F. Falzon, E. A. Markatou, Z. Espiritu, and R. Tamassia. Range search over encrypted multi-attribute data. *Proceedings of the VLDB Endowment*, 16(4), 2022.
- [25] R. A. Finkel and J. L. Bentley. Quad trees a data structure for retrieval on composite keys. *Acta informatica*, 4:1–9, 1974.
- [26] M. J. Freedman, Y. Ishai, B. Pinkas, and O. Reingold. Keyword search and oblivious pseudorandom functions. In *Theory of Cryptography Conference*, pages 303–324. Springer, 2005.
- [27] B. Fuller, M. Varia, A. Yerukhimovich, E. Shen, A. Hamlin, V. Gadepally, R. Shay, J. D. Mitchell, and R. K. Cunningham. SoK: Cryptographically protected database search. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 172–191. IEEE, 2017.
- [28] M. George, S. Kamara, and T. Moataz. Structured encryption and dynamic leakage suppression. In *Advances in Cryptology–EUROCRYPT 2021: 40th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, October 17–21, 2021, Proceedings, Part III*, pages 370–396. Springer, 2021.
- [29] N. Gilboa and Y. Ishai. Distributed point functions and their applications. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 640–658. Springer, 2014.
- [30] M. Grabisch, J.-L. Marichal, R. Mesiar, and E. Pap. *Aggregation functions*, volume 127. Cambridge University Press, 2009.
- [31] P. Grubbs, M.-S. Lacharité, B. Minaud, and K. G. Paterson. Pump up the volume: Practical database reconstruction from volume leakage on range queries. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 315–331, 2018.
- [32] P. Grubbs, K. Sekniqi, V. Bindschaedler, M. Naveed, and T. Ristenpart. Leakage-abuse attacks against order-revealing encryption. In *Security and Privacy (SP), 2017 IEEE Symposium on*, pages 655–672. IEEE, 2017.
- [33] Z. Gui, K. G. Paterson, and S. Patranabis. Rethinking searchable symmetric encryption. In *IEEE Security and Privacy*, 2023.
- [34] A. Guttman. R-trees: A dynamic index structure for spatial searching. In *Proceedings of the 1984 ACM SIGMOD international conference on Management of data*, pages 47–57, 1984.
- [35] J. Ha, C. Cachet, L. Demarest, S. Ahmad, and B. Fuller. Private eyes: Zero-leakage iris searchable encryption. In *Proceedings of the Fifteenth ACM Conference on Data and Application Security and Privacy*, pages 383–394, 2024.
- [36] T. Hackenjos, F. Hahn, and F. Kerschbaum. Sagma: secure aggregation grouped by multiple attributes. In *Proceedings of the 2020 ACM SIGMOD international conference on management of data*, pages 587–601, 2020.
- [37] M. Heidari-pour, L. Kian, M. Rezapour, M. Holcomb, B. Fuller, G. Agrawal, and H. Maleki. Organizing records for retrieval in multi-dimensional range searchable encryption. In *Proceedings of the 21st International Conference on Security and Cryptography - Volume 1: SECRYPT*, pages 459–466. INSTICC, SciTePress, 2024.
- [38] C.-T. Ho, R. Agrawal, N. Megiddo, and R. Srikant. Range queries in olap data cubes. *ACM SIGMOD Record*, 26(2):73–88, 1997.
- [39] N. Ibtehaz, M. Kaykobad, and M. S. Rahman. Multidimensional segment trees can do range queries and updates in logarithmic time. *arXiv preprint arXiv:1811.01226*, 2018.

- [40] N. Ibtihaz, M. Kaykobad, and M. Sohel Rahman. Multidimensional segment trees can do range updates in poly-logarithmic time. *Theoretical Computer Science*, 854:30–43, Jan. 2021.
- [41] M. S. Islam, M. Kuzu, and M. Kantarcioglu. Access pattern disclosure on searchable encryption: ramification, attack and mitigation. In *NDSS*, volume 20, page 12. Citeseer, 2012.
- [42] S. Kamara, A. Kati, T. Moataz, T. Schneider, A. Treiber, and M. Yonli. Sok: Cryptanalysis of encrypted search with leaker - a framework for leakage attack evaluation on real-world data. In *Euro S&P*, 2022. <https://ia.cr/2021/1035>.
- [43] G. Kellaris, G. Kollios, K. Nissim, and A. O’Neill. Generic attacks on secure outsourced databases. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, pages 1329–1340, 2016.
- [44] E. M. Kornaropoulos and P. Efstathopoulos. The case of adversarial inputs for secure similarity approximation protocols. In *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 247–262. IEEE, 2019.
- [45] E. M. Kornaropoulos, C. Papamanthou, and R. Tamassia. Data recovery on encrypted databases with k-nearest neighbor query leakage. In *2019 IEEE Symposium on Security and Privacy (SP)*, pages 1033–1050. IEEE, 2019.
- [46] E. M. Kornaropoulos, C. Papamanthou, and R. Tamassia. The state of the uniform: attacks on encrypted databases beyond the uniform query distribution. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 1223–1240. IEEE, 2020.
- [47] E. Kushnir, G. Moshkovich, and H. Shaul. Secure range-searching using copy-and-recurse. *Cryptology ePrint Archive*, 2023.
- [48] A. Lazzaretti, Z. Liu, B. Fisch, and C. Papamanthou. Multi-server doubly efficient pir. *Cryptology ePrint Archive*, 2024.
- [49] H. Li, S. Navot, and S. Tessaro. POPSTAR: Lightweight threshold reporting with reduced leakage. In *USENIX Security 24*, pages 6939–6956, 2024.
- [50] W.-K. Lin, E. Mook, and D. Wichs. Doubly efficient private information retrieval and fully homomorphic ram computation from ring lwe. In *Proceedings of the 55th Annual ACM Symposium on Theory of Computing*, STOC 2023, page 595–608, New York, NY, USA, 2023. Association for Computing Machinery.
- [51] Y. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence*, 42(4):824–836, 2018.
- [52] Y. Manolopoulos, A. Nanopoulos, A. N. Papadopoulos, and Y. Theodoridis. *R-Trees: Theory and Applications: Theory and Applications*. Springer Science & Business Media, 2006.
- [53] E. A. Markatou and R. Tamassia. Full database reconstruction with access and search pattern leakage. In *International Conference on Information Security*, pages 25–43. Springer, 2019.
- [54] J. Matoušek. Efficient partition trees. In *Proceedings of the seventh annual symposium on Computational geometry*, pages 1–9, 1991.
- [55] P. Mishra. A new algorithm for updating and querying sub-arrays of multidimensional arrays. *arXiv preprint arXiv:1311.6093*, 2013.
- [56] D. Mouris, C. Patton, H. Davis, P. Sarkar, and N. G. Tsoutsos. Mastic: Private weighted heavy-hitters and attribute-based metrics. *Proceedings on Privacy Enhancing Technologies*, 2025.

- [57] D. Mouris, P. Sarkar, and N. G. Tsoutsos. Plasma: Private, lightweight aggregated statistics against malicious adversaries. *Proceedings on Privacy Enhancing Technologies*, 2024(3), 2024.
- [58] H. Okada, R. Player, S. Pohmann, and C. Weinert. Towards practical doubly-efficient private information retrieval. In *International Conference on Financial Cryptography and Data Security*, pages 264–282. Springer, 2024.
- [59] S. Pia, A. Appan, M. Rezapour, A. Shukla, N. Date, B. Fuller, L. Ren, and D. Heath. Oblivious single access machines are concretely efficient. *Cryptology ePrint Archive*, Paper 2026/451, 2026.
- [60] F. P. Preparata and M. I. Shamos. *Computational geometry: an introduction*. Springer Science & Business Media, 2012.
- [61] K. Ren, Y. Guo, J. Li, X. Jia, C. Wang, Y. Zhou, S. Wang, N. Cao, and F. Li. Hybridx: New hybrid index for volume-hiding range queries in data outsourcing services. In *2020 IEEE 40th international conference on distributed computing systems (ICDCS)*, pages 23–33. IEEE, 2020.
- [62] M. Rezapour and B. Fuller. Proxcode: Efficient proximity searchable encryption from error correcting codes. In *CT-RSA*, 2026.
- [63] D. S. Roche, A. Aviv, and S. G. Choi. A practical oblivious map data structure with secure deletion and history independence. In *2016 IEEE Symposium on Security and Privacy (SP)*, pages 178–197. IEEE, 2016.
- [64] F. Rusu. Multidimensional array data management. *Foundations and Trends® in Databases*, 12(2-3):69–220, 2023.
- [65] M. Shekelyan, A. Dignös, and J. Gamper. Sparse prefix sums: constant-time range sum queries over sparse multidimensional data cubes. *Information Systems*, 82:136–147, 2019.
- [66] S. Shirotake and K. Shimizu. Efficient privacy preserving range query using segment tree. In *2024 58th Annual Conference on Information Sciences and Systems (CISS)*, pages 1–6. IEEE, 2024.
- [67] E. Stefanov, M. V. Dijk, E. Shi, T.-H. H. Chan, C. Fletcher, L. Ren, X. . Yu, and S. Devadas. Path oram: An extremely simple oblivious ram protocol. *J. ACM*, 65(4), apr 2018.
- [68] B. H. M. Tan, H. T. Lee, H. Wang, S. Ren, and K. M. M. Aung. Efficient private comparison queries over encrypted databases using fully homomorphic encryption with finite fields. *IEEE Transactions on Dependable and Secure Computing*, 18(6):2861–2874, 2020.
- [69] G. Wang, C. Liu, Y. Dong, H. Pan, P. Han, and B. Fang. Query recovery attacks on searchable encryption based on partial knowledge. In *International Conference on Security and Privacy in Communication Systems*, pages 530–549. Springer, 2017.
- [70] X. S. Wang, K. Nayak, C. Liu, T. H. Chan, E. Shi, E. Stefanov, and Y. Huang. Oblivious data structures. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, pages 215–226, 2014.
- [71] Y. Wang, A. Nandi, and G. Agrawal. Saga: Array storage as a db with support for structural aggregations. In *Proceedings of the 26th international conference on scientific and statistical database management*, pages 1–12, 2014.
- [72] Z. Ye, X. Wang, Z. Liu, D. Zhu, and J. Ma. Obir-tree: An efficient oblivious index for spatial keyword queries on secure enclaves. *Proceedings of the ACM on Management of Data*, 3(1):1–24, 2025.
- [73] J. Zhu, L. Patel, M. Zaharia, and R. A. Popa. Compass: Encrypted semantic search with high accuracy. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 915–938, 2025.

Dataset	Theo.Max	Exp.Max
book 1D	4	4
gowalla 1D	4	4
spitz 2D	76	56
cali 2D	76	60
gowalla 2D-50K	92	60
gowalla 2D-100K	100	72
synthetic 2D-2048-d	84	72
synthetic 2D-2048-sp	84	76
synthetic 2D-1024-sp	76	68
nh 3D	444	228
gowalla 3D	524	156
synthetic 3D	628	328
synthetic 3D	844	464

Table 10: Comparison of theoretical and experimental maximum number of accesses per level of the multi-dimensional segment tree.

A Maximum Number of Accesses

We measured the maximum number of accesses per level of the segment tree through experiments on 10,000 queries and calculated the theoretical maximum accesses at each level for each dataset using the formula

$$\text{MAX}_{\text{Theory}} = 4 \left(1 + \sum_{i=1}^{d-1} 2^i \prod_{j=1}^i (\log(n_j) - 1) \right),$$

with the results presented in Table 10.